# AADK 10: Task 3 — Mini Project: Compose App Feature Combining OOP and Lambdas

**Student Name:** Rajat Prakash Dhal

**Course/Unit:** Android Development with Kotlin – **Session 10**

**USN: 1hk22cs115**

**email:** 1hk22cs115@hkbk.edu.in

### 1. App Theme

**Chosen Theme: Habit Tracker App**

The Habit Tracker app helps users track daily habits such as exercising, reading, or studying. Each habit is represented as an object containing information like habit name, category, completion status, and streak count. Users can mark habits as completed using interactive buttons in the UI. Lambdas are used to toggle habit completion and filter habits based on their status.

---

### 2. Class Structure and Data

**Main Class**

```
class Habit(
    val name: String,
    val category: String,
    var streak: Int,
    var isCompleted: Boolean
) {

    fun toggleCompletion() {
        isCompleted = !isCompleted
    }
```

```
    fun increaseStreak() {
        streak++
    }

    companion object {
        const val MAX_STREAK = 365
    }
}
```

**Properties**

- name: String
- category: String
- streak: Int
- isCompleted: Boolean

**Methods**

- toggleCompletion() – toggles habit completion status
- increaseStreak() – increases streak count

**Companion Object**

- MAX_STREAK constant representing the maximum streak limit.

---

**List of Habit Objects**

```
val habits = listOf(
    Habit("Morning Exercise", "Health", 5, false),
    Habit("Read Book", "Learning", 12, true),
    Habit("Meditation", "Wellness", 7, false),
    Habit("Practice Kotlin", "Programming", 3, false)
)
```

---

**Higher-Order Function with Lambda**

```
fun filterHabits(
    habits: List<Habit>,
    condition: (Habit) -> Boolean
): List<Habit> {
    return habits.filter(condition)
}
```

**Example Usage**

```
val completedHabits = filterHabits(habits) { it.isCompleted }
```

This lambda filters only the habits that are marked as completed.

---

**3. Compose UI Integration Plan**

**UI Structure**

The app UI will display habits using **LazyColumn**. Each habit item will contain:

- Habit name
- Category
- Current streak
- Toggle completion button

**Example Compose Snippet**

```
LazyColumn {
    items(habits) { habit ->
        Row {
            Text(habit.name)

            Button(onClick = { habit.toggleCompletion() }) {
                Text("Toggle")
            }
```

```
        }
    }
}
```

**UI Response**

- When the **Toggle button** is clicked, the lambda executes toggleCompletion().
- The habit's isCompleted state changes.
- Compose recomposes the UI automatically and updates the display.

---

**4. Flow Diagram and Architecture**

**Data Flow**

Habit Class (Data Model)

↓

List of Habit Objects

↓

Lambda Filter Function

↓

Compose LazyColumn UI

↓

User Interaction (Button Click)

↓

Lambda → toggleCompletion()

↓

State Update → UI Recomposition

**Responsibilities**

| Component | Responsibility |
|---|---|
| Habit Class | Data model & business logic |
| Lambda Function | Filtering or transforming habit list |
| Compose UI | Rendering list and handling user interaction |
| State Management | Updating UI when data changes |

---

**5. Error & Edge Case Planning**

**Edge Case 1 — Empty Habit List**

Problem: No habits available to display.

Handling:

```
if (habits.isEmpty()) {
    Text("No habits available")
}
```

---

**Edge Case 2 — Null Habit Object**

Problem: Data could be null if fetched from database.

Handling:
Use **safe call operator**

```
habit?.toggleCompletion()
```

---

**Edge Case 3 — Invalid User Input**

Problem: User creates a habit with empty name.

Handling:
Validate before creating object.

```
if (name.isBlank()) {
    println("Habit name cannot be empty")
```

```
}
```

---

**6. Reflection & Improvement Plan**

This project helped combine object-oriented programming and functional programming concepts. The Habit class models the data and behavior, while lambdas provide flexible ways to filter and manipulate habit lists. Using lambdas makes the code shorter and easier to reuse. In Jetpack Compose, lambdas simplify event handling such as button clicks and state updates.

One trade-off between OOP and functional styles is that OOP focuses on structured data models, while functional programming focuses on behavior and transformations. Combining both provides a balanced and flexible design.

If this were developed into a real app feature, improvements could include persistent storage using Room database, better state management with ViewModel, and more advanced filtering options such as sorting habits by streak or category.